厦门大学信息学院 本科选修课
2024-2025 第二学期

模 式 识 别

Python动手课（深度学习篇）

主 讲：王程教授、陈龙彪副教授

厦门大学信息学院计算机科学与技术系

人工智能引论101计划系列课程



课程目标

- 理解深度学习的基本概念。
- 初探深度学习工具库Theano、TensorFlow和Keras。
- 掌握使用Keras构建神经网络，尝试用Python深度学习库完成实践项目。

课程概览

- 深度学习工具库： Theano, TensorFlow, Keras
- 多层感知机
- 网络训练技巧
- 卷积神经网络实验
- 数据增强
- 循环神经网络实验

1 深度学习库

Theano

- Theano
  - Theano，希腊神话中代表思想和思维的女神
  - Theano是一个用于定义、优化和评估数学表达式的开源库。它提供了一种高效的方式来进行数值计算，特别是针对大规模的数学运算，如矩阵运算。
  - Theano是由蒙特利尔大学的MILA实验室开发的，于2007年发布，被认为是深度学习框架的先驱之一。



Theano

- 安装

```
sudo pip install Theano
```

- 示例

```
# Example of Theano library
import theano
from theano import tensor
# declare two symbolic floating-point scalars
a = tensor.dscalar()
b = tensor.dscalar()
# create a simple symbolic expression
c = a + b
# convert the expression into a callable object that takes (a,b) and computes c
f = theano.function([a,b], c)
# bind 1.5 to 'a', 2.5 to 'b', and evaluate 'c'
result = f(1.5, 2.5)
print(result)
```

TensorFlow

TensorFlow

- TensorFlow是由Google开发的开源机器学习框架，于2015年首次发布。
- TensorFlow提供了一个灵活的基础架构，用于构建和训练各种机器学习模型，包括深度学习模型。
- TensorFlow提供了类似于Theano的功能，但它更加灵活，支持分布式计算、移动端部署等功能。TensorFlow也被广泛应用于学术界和工业界。



TensorFlow

- 安装

```
sudo pip install TensorFlow
```

- 示例

```
# Example of TensorFlow library
import tensorflow as tf
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
# declare two symbolic floating-point scalars
a = tf.placeholder(tf.float32)
b = tf.placeholder(tf.float32)
# create a simple symbolic expression using the add function
add = tf.add(a, b)
# bind 1.5 to 'a', 2.5 to 'b', and evaluate 'c'
sess = tf.Session()
binding = {a: 1.5, b: 2.5}
c = sess.run(add, feed_dict=binding)
print(c)
```

Keras

Keras

- Keras是一个高级神经网络API，最初由 François Chollet 开发。
- Keras最初是建立在Theano之上的，后来也支持TensorFlow等其他后端。
- Keras的设计目标是让用户能够快速、简单地构建神经网络模型。它提供了高级的抽象和简洁的API，使得用户可以轻松地定义各种类型的神经网络，包括卷积神经网络、循环神经网络等。



Keras

安装

```
sudo pip install keras
```

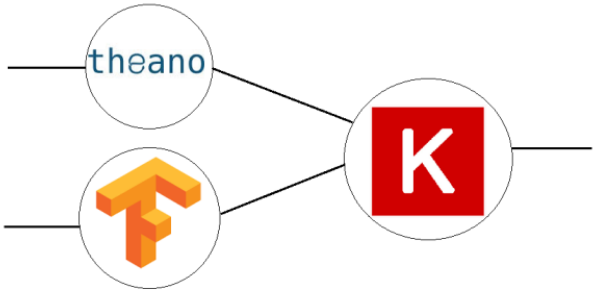
模型构建

- Keras深度学习模型的构建包括定义（Define）、编译（Compile）和拟合（Fit），然后可以进行评估（Evaluation）或用来进行预测（Predict）。

1. **Define your model**. Create a `Sequential` model and add configured layers.
2. **Compile your model**. Specify loss function and optimizers and call the `compile()` function on the model.
3. **Fit your model**. Train the model on a sample of data by calling the `fit()` function on the model.
4. **Make predictions**. Use the model to generate predictions on new data by calling functions such as `evaluate()` or `predict()` on the model.

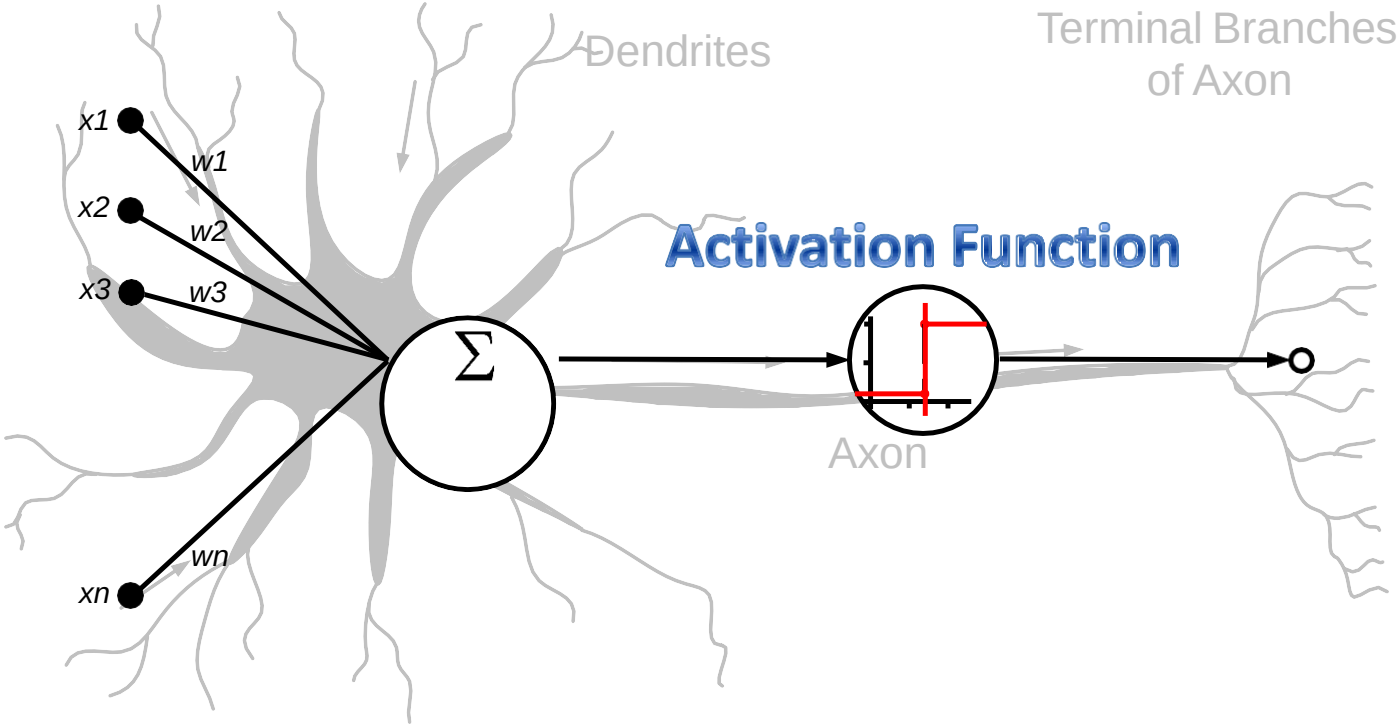
Theano、TensorFlow和Keras

- 在早期，Keras是建立在Theano之上的，但后来随着TensorFlow的崛起，Keras被重构为一个能够在多种深度学习后端（包括TensorFlow）上运行的高级API。
- TensorFlow在内部集成了Keras，因此使用TensorFlow时，可以直接通过TensorFlow的API来使用Keras。
- Theano虽然在一段时间内被广泛使用，但随着TensorFlow的发展，其在深度学习社区中的影响力逐渐减弱，最终于2017年被宣布不再积极开发。
- 综上所述，TensorFlow是一个强大的深度学习框架，Keras是其高级API，而Theano则是一个早期的深度学习框架，其部分功能被后来的框架所取代。



2 多层感知机（MLP）

人工神经元


Dendrites
Terminal Branches of Axon
x1
x2
x3
xn
w1
w2
w3
wn
Σ
Activation Function
Axon

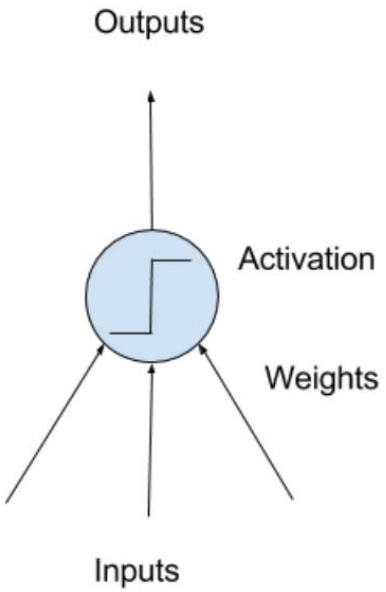
MLP

MLP

- MLP(多层感知机Multilayer Perceptron)是一种基本的人工神经网络结构。
- MLP由多个神经元层组成，每个神经元层与下一层全连接。

Weights

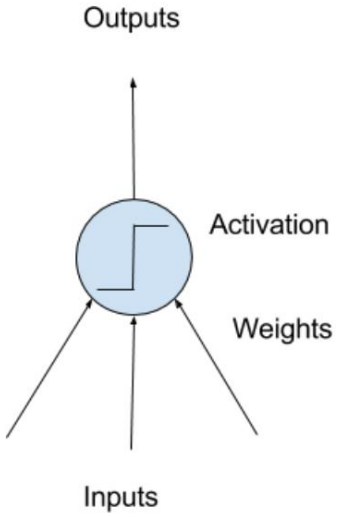
- 权重表示了连接的强度，它决定了上一层神经元的输出对下一层神经元的影响程度。

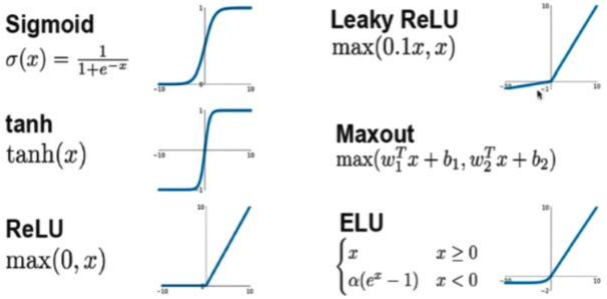


MLP

Activation

- 激活函数是将加权输入的总和简单映射到神经元的输出的过程。
- 它控制着神经元被激活的阈值和输出信号的强度。
- **ReLU**：线性激活函数，适用于隐藏层。
- **Sigmoid**：S形曲线，适用于输出层，特别是在二分类问题中。
- **Tanh**：双曲正切函数，与sigmoid类似，但输出范围是-1到1。

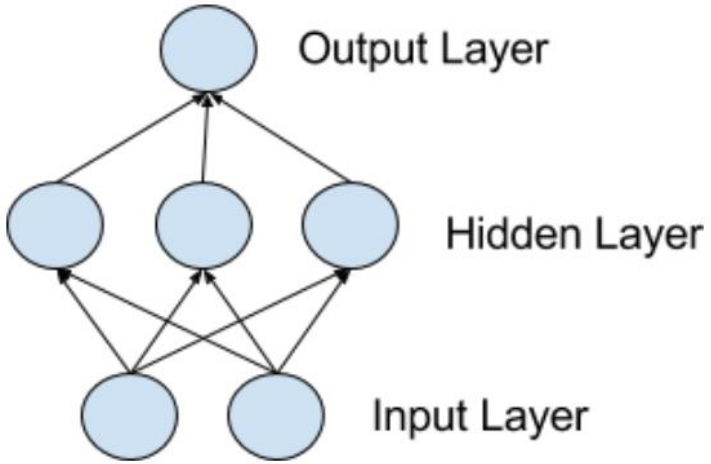




MLP

神经元网络

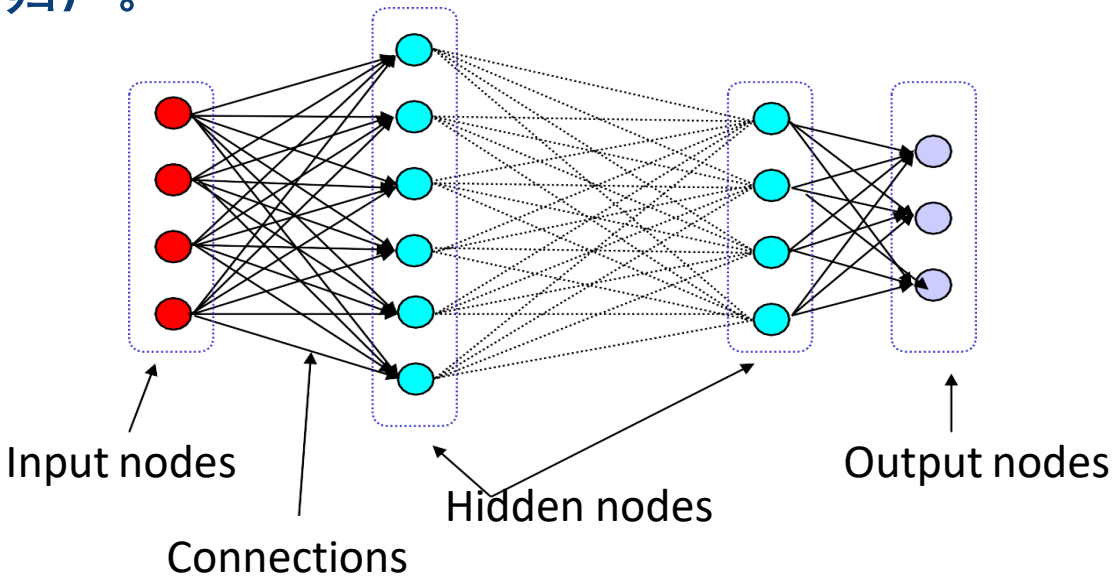
- 神经元被排列成神经元网络。一排神经元被称为一层，一个网络可以有多层。网络中神经元的体系结构通常被称为网络拓扑结构。
- Input Layer
- Hidden Layer
- Output Layer



MLP

工作原理

- 输入信号通过输入层传递给网络。
- 输入信号与每个连接的权重相乘，然后加上偏置（bias）。
- 加权和通过激活函数进行非线性变换，生成每个神经元的输出。
- 输出传递到下一层，直到达到输出层。
- 输出层通常采用不同的激活函数，具体取决于任务类型（如分类或回归）。



MLP训练网络

- 1 数据准备
  - 数据必须数字化，且通常是数值型。
  - 分类数据需要进行独热编码（One-Hot Encoding）。
  - 输入数据需要标准化或归一化处理。
- 2 随机梯度下降（SGD）
  - SGD是用于最小化损失函数的优化算法。通过计算损失函数关于权重的梯度来更新权重。
  - 反向传播包括两个阶段：前向传播和误差反向传播。前向传播计算预测输出。误差反向传播调整权重以减少预测误差。
- 3 权重更新
  - 权重通过学习率乘以权重的梯度来更新。学习率控制权重更新的步长。
  - 动量(Momentum)帮助权重更新保持一致的方向，避免震荡。
  - 自适应学习率算法（如Adam优化器）根据梯度的历史调整学习率。

MLP实践

使用Keras实现MLP

1. Load Data.
2. Define Model.
3. Compile Model.
4. Fit Model.
5. Evaluate Model.
6. Make Predictions.

皮马印第安人糖尿病发病预测模型（二分类）

```python
# first neural network with keras tutorial
from numpy import loadtxt
from keras.models import Sequential
from keras.layers import Dense
# load the dataset
dataset = loadtxt('pima-indians-diabetes.csv', delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
# define the keras model
model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# compile the keras model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# fit the keras model on the dataset
model.fit(X, y, epochs=150, batch_size=16)
# evaluate the keras model
_, accuracy = model.evaluate(X, y)
print('Accuracy: %.2f' % (accuracy*100))
```

3 网络训练技巧

Save & Load

- Save and Load Keras model weights to HDF5 formatted files.
- Save and load Keras model structure to JSON files.
- Save and load Keras model structure to YAML files.

Save示例

```
# serialize model to JSON
model_json = model.to_json()
with open("model.json", "w") as json_file:
    json_file.write(model_json)
# serialize weights to HDF5
model.save_weights("model.h5")
print("Saved model to disk")
```

```
# serialize model to YAML
model_yaml = model.to_yaml()
with open("model.yaml", "w") as yaml_file:
    yaml_file.write(model_yaml)
# serialize weights to HDF5
model.save_weights("model.h5")
print("Saved model to disk")
```

Load示例

```
# load json and create model
json_file = open('model.json', 'r')
loaded_model_json = json_file.read()
json_file.close()
loaded_model = model_from_json(loaded_model_json)
# load weights into new model
loaded_model.load_weights("model.h5")
print("Loaded model from disk")
```

```
# load YAML and create model
yaml_file = open('model.yaml', 'r')
loaded_model_yaml = yaml_file.read()
yaml_file.close()
loaded_model = model_from_yaml(loaded_model_yaml)
# load weights into new model
loaded_model.load_weights("model.h5")
print("Loaded model from disk")
```

Checkpoint

- Checkpoint通常指的是在训练模型时定期保存模型的参数或状态的操作。主要目的是在训练过程中防止意外中断导致模型参数丢失，并且可以在需要时恢复模型的训练状态。

Save：

```
# checkpoint
filepath="weights-improvement-{epoch:02d}-{val_accuracy:.2f}.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='val_accuracy', verbose=1, save_best_only=True, mode='max')
callbacks_list = [checkpoint]
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10, callbacks=callbacks_list, verbose=0)
```

Load：

```
# load weights
model.load_weights("weights.best.hdf5")
# Compile model (required to make predictions)
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
print("Created model and loaded weights from file")
```

工作目录中可以看到一些包含HDF5格式的网络权重的文件。例如：

```
...
weights-improvement-53-0.76.hdf5
weights-improvement-71-0.76.hdf5
weights-improvement-77-0.78.hdf5
weights-improvement-99-0.78.hdf5
```

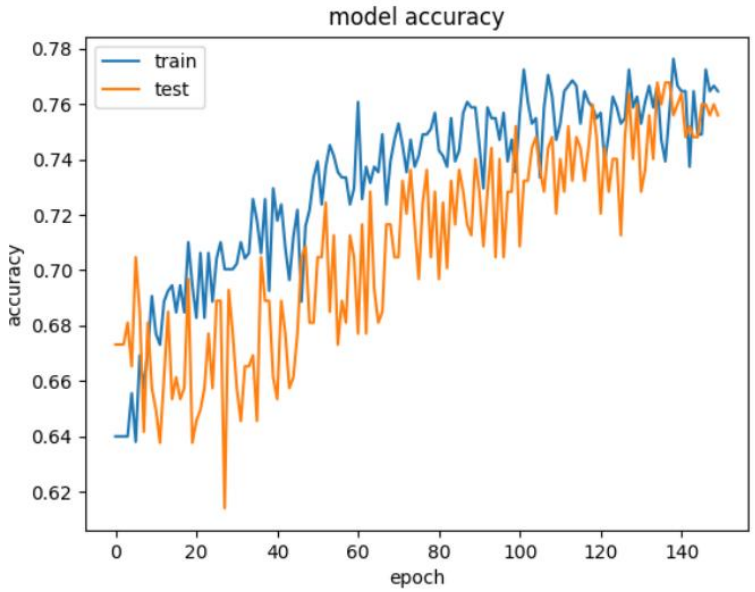
训练过程可视化

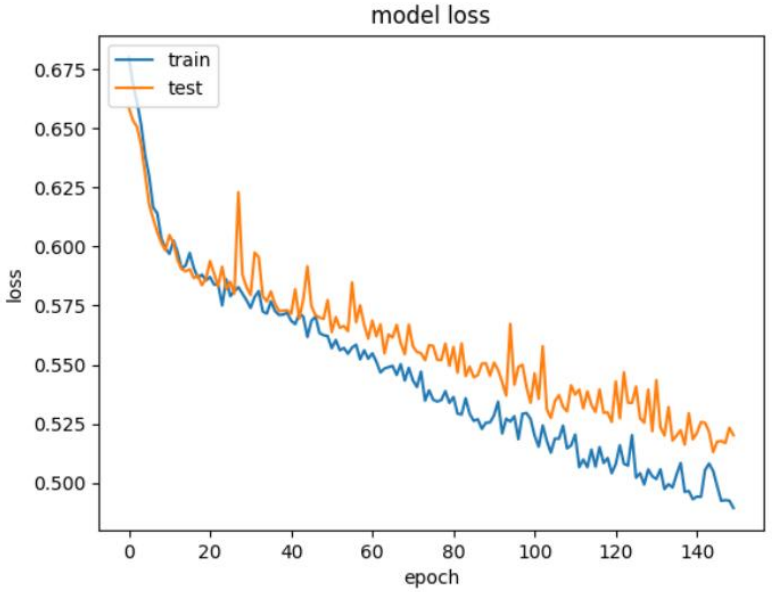
- 指标记录
- 指标可视化

```
# Fit the model
history = model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10, verbose=0)
# list all data in history
print(history.history.keys())
# summarize history for accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
# summarize history for loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

训练过程可视化

- 指标记录
- 指标可视化





Dropout正则化

- Dropout正则化
  - 是一种用于减少深度神经网络过拟合的技术。在训练过程中，Dropout随机地忽略一部分神经元的输出，即将它们的权重置为零，从而使得网络在训练期间表现得更加鲁棒，并减少对某些特定神经元的依赖。
- 使用Dropout时的一些建议
  - 丢弃概率宜选择较小的范围，通常在神经元数量的20%到50%之间，其中20%是一个较为适宜的起点。概率过低几乎不会起作用，而过高则可能导致网络学习不足。
  - 建议采用规模较大的网络结构。在较大的网络上使用Dropout，通常能够获得更好的性能表现，这样模型就有更多机会学习到独立的特征表示。
  - 建议在输入层和隐藏层均应用Dropout。实践表明，在神经网络的每一层都使用Dropout可以取得良好的效果。
  - 使用较大的学习率，并结合衰减和较大的动量。将学习率调高10到100倍，并采用较高的动量值，如0.9或0.99。
  - 对网络权重的大小进行适当约束。较大的学习率可能导致网络权重增长过快。因此，通过设置最大范数正则化约束，如设置为4或5，可以改善训练效果。

Dropout正则化

Baseline

```
# baseline
def create_baseline():
    # create model
    model = Sequential()
    model.add(Dense(60, input_dim=60, activation='relu'))
    model.add(Dense(30,  activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    sgd = SGD(lr=0.01, momentum=0.8)
    model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
    return model
```

Use dropout
on input layers

```
# dropout in the input layer with weight constraint
def create_model():
    # create model
    model = Sequential()
    model.add(Dropout(0.2, input_shape=(60,)))
    model.add(Dense(60, activation='relu', kernel_constraint=maxnorm(3)))
    model.add(Dense(30, activation='relu', kernel_constraint=maxnorm(3)))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    sgd = SGD(lr=0.1, momentum=0.9)
    model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
    return model
```

Use dropout
on hidden layers

```
# dropout in hidden layers with weight constraint
def create_model():
    # create model
    model = Sequential()
    model.add(Dense(60, input_dim=60, activation='relu', kernel_constraint=maxnorm(3)))
    model.add(Dropout(0.2))
    model.add(Dense(30, activation='relu', kernel_constraint=maxnorm(3)))
    model.add(Dropout(0.2))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    sgd = SGD(lr=0.1, momentum=0.9)
    model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
    return model
```

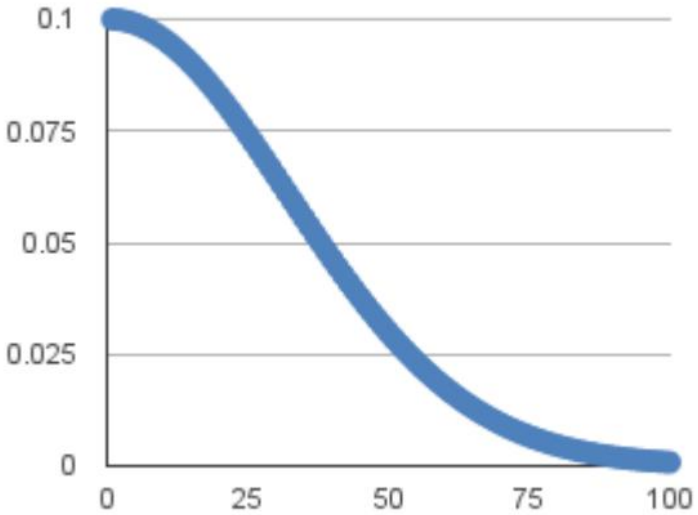
Learning Rate Schedules

- Learning Rate Schedule
  - 学习率调度（Learning Rate Schedules）是深度学习中的一种技术，用于动态调整模型训练过程中的学习率。学习率是指在优化算法（如随机梯度下降）中控制参数更新步长的参数，它直接影响着模型的收敛速度和性能。
  - 学习率调度的主要目的是在训练过程中逐步减小学习率，从而更好地探索和优化模型的参数空间。
- 常见的学习率调度策略包括：
  - 固定学习率（Fixed Learning Rate）：在整个训练过程中保持不变的学习率。虽然简单直接，但可能会导致训练过程中的性能波动或收敛速度过慢。
  - 学习率衰减（Learning Rate Decay）：在训练的每个epoch或特定时间间隔内，将学习率按照一定的比例进行衰减，通常是乘以一个衰减因子。
  - 余弦退火（Cosine Annealing）：学习率按照余弦函数的形式进行周期性调整。在每个周期结束时，学习率被重新初始化为初始学习率的一部分，并且在下一个周期内以余弦函数的形式逐渐减小。
  - 学习率预热（Learning Rate Warmup）：在训练的初始阶段，先使用较小的学习率进行模型的预热，然后再逐渐增加到设定的学习率。
  - 自适应学习率方法（Adaptive Learning Rate Methods）：使用自适应的方式动态调整学习率，如Adam、Adagrad、RMSProp等优化算法。这些算法通常根据参数的梯度大小和更新历史来自动调整学习率。

Learning Rate Schedules

Time-Based Learning Rate Schedule

$$LearningRate = LearningRate \times \frac{1}{1 + decay \times epoch}$$

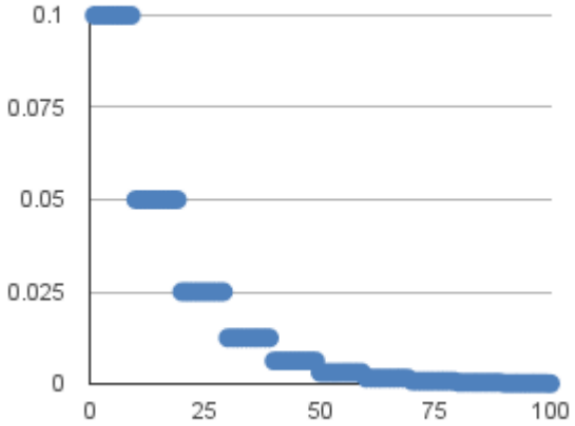



```
epochs = 50
learning_rate = 0.1
decay_rate = learning_rate / epochs
momentum = 0.8
sgd = SGD(lr=learning_rate, momentum=momentum, decay=decay_rate, nesterov=False)
model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=epochs, batch_size=28, verbose=2)
```

Learning Rate Schedules

Drop-Based Learning Rate Schedule

$$LearningRate = InitialLearningRate \times DropRate^{floor(\frac{1+Epoch}{EpochDrop})}$$




```
# learning rate schedule
def step_decay(epoch):
  initial_lrate = 0.1
  drop = 0.5
  epochs_drop = 10.0
  lrate = initial_lrate * math.pow(drop, math.floor((1+epoch)/epochs_drop))
  return lrate

# Compile model
sgd = SGD(lr=0.0, momentum=0.9)
model.compile(loss='binary_crossentropy', optimizer=sgd, metrics=['accuracy'])
# learning schedule callback
lrate = LearningRateScheduler(step_decay)
callbacks_list = [lrate]
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=50, batch_size=28, callbacks=callbacks_list,
    verbose=2)
```

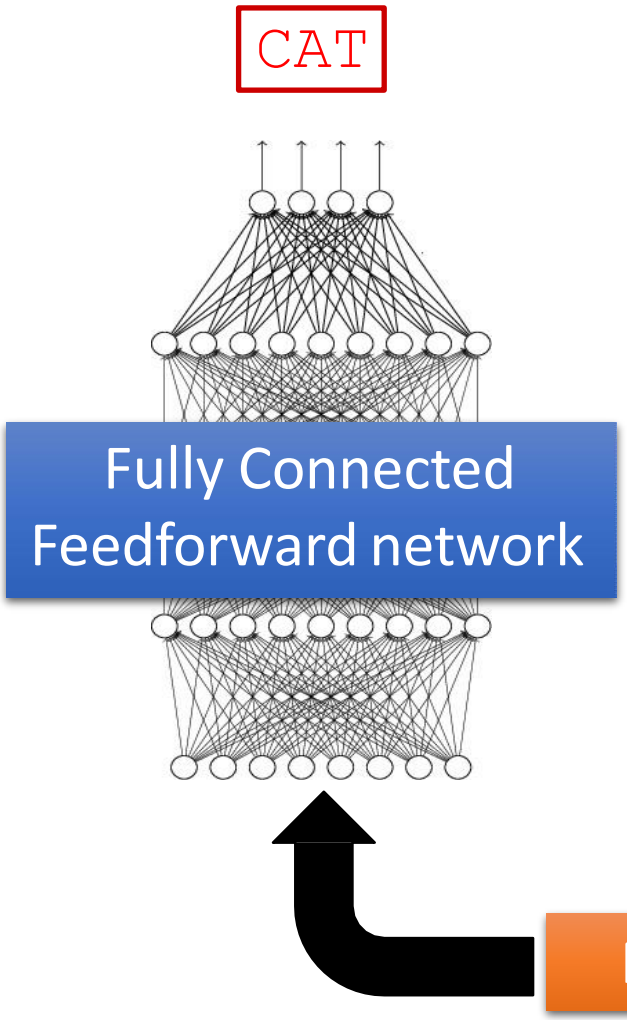
Learning Rate Schedules

- 使用神经网络时考虑学习率调度的一些技巧
  - **增加初始学习率**。由于学习率会逐渐减小，因此建议从较大的值开始，以便从中逐渐减小。较大的学习率会导致权重发生更大的变化，至少在开始阶段，这有助于后续的微调。
  - **使用较大的动量**。较大的动量值有助于优化算法在学习率减小至较小值时继续朝着正确的方向进行更新。
  - **尝试不同的调度方案**。不同的学习率调度方案对于不同的问题可能有不同的效果，因此建议尝试几种不同配置选项，并观察哪种对您的问题效果最好。还可以尝试指数变化的调度方案，甚至根据模型在训练集或测试集上的准确性进行调整的方案。

4 卷积神经网络(CNN)

CNN网络结构




CAT
Fully Connected
Feedforward network
Convolution
Max Pooling
Convolution
Max Pooling
Flatten
可以重复
多次

CNN

- 卷积层（Convolutional Layer）
  - CNN通过卷积操作来提取输入数据的特征。卷积操作是指通过滑动一个滤波器（也称为卷积核）在输入数据上进行局部加权求和，从而得到输出特征图。卷积层可以有效地提取局部特征，并具有平移不变性，使得模型对于输入数据的平移、旋转等变换具有一定的鲁棒性。
- 池化层（Pooling Layer）
  - 池化层通常紧跟在卷积层之后，用于减少特征图的尺寸和参数数量，从而降低模型复杂度和计算量。
- 全连接层（Fully Connected Layer）
  - 在CNN的末尾，通常会添加一个或多个全连接层，用于将卷积和池化层提取的特征进行整合，并生成最终的输出。全连接层通常与softmax函数结合，用于多分类任务的输出。

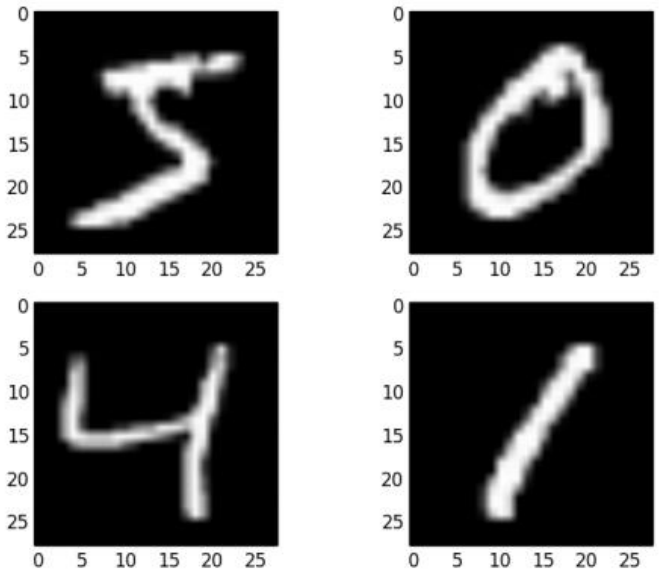
基于Keras的手写数字识别

介绍

- 手写数字识别就像是‘Hello World’一样，是开始探索机器学习和深度学习的世界的第一步。

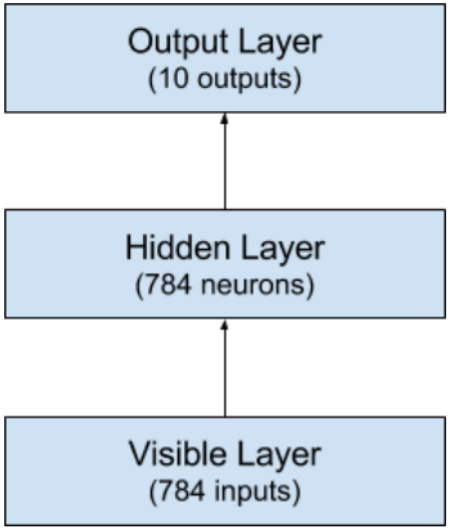
数据集

- MNIST：包含了大约70000张28x28像素的灰度图像，这些图像显示了从0到9的手写数字。



基于MLP的手写数字识别

```python
# Baseline MLP for MNIST dataset
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import np_utils
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# flatten 28*28 images to a 784 vector for each image
num_pixels = X_train.shape[1] * X_train.shape[2]
X_train = X_train.reshape((X_train.shape[0], num_pixels)).astype('float32')
X_test = X_test.reshape((X_test.shape[0], num_pixels)).astype('float32')
# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
# one hot encode outputs
y_train = np_utils.to_categorical(y_train)
y_test = np_utils.to_categorical(y_test)
num_classes = y_test.shape[1]
# define baseline model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(num_pixels, input_dim=num_pixels, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# build the model
model = baseline_model()
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=10, batch_size=200, verbose=2)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Baseline Error: %.2f%%" % (100-scores[1]*100))
```

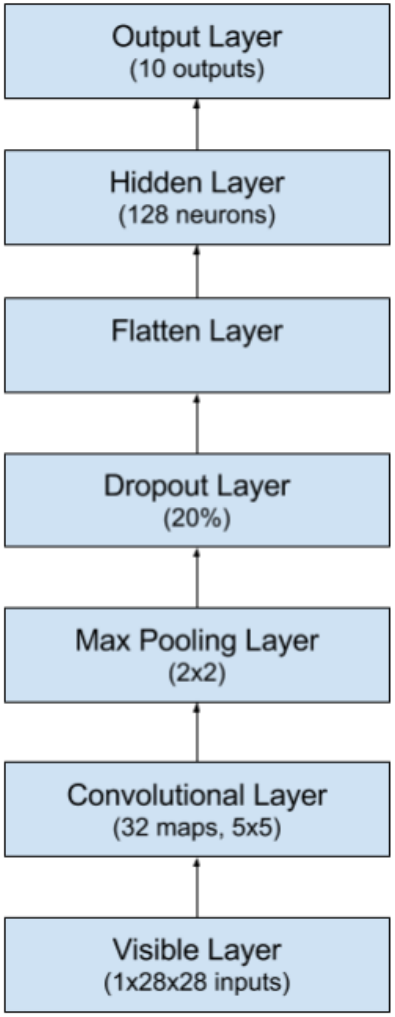


Baseline Model with Multilayer Perceptrons

基于CNN的手写数字识别

```python
# Simple CNN for the MNIST Dataset
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Dropout
from keras.layers import Flatten
from keras.layers.convolutional import Conv2D
from keras.layers.convolutional import MaxPooling2D
from keras.utils import np_utils
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][width][height][channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1)).astype('float32')
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1)).astype('float32')
# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
# one hot encode outputs
y_train = np_utils.to_categorical(y_train)
y_test = np_utils.to_categorical(y_test)
num_classes = y_test.shape[1]
# define a simple CNN model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Conv2D(32, (5, 5), input_shape=(28, 28, 1), activation='relu'))
    model.add(MaxPooling2D())
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dense(num_classes, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# build the model
model = baseline_model()
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=10, batch_size=200)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("CNN Error: %.2f%%" % (100-scores[1]*100))

```



Convolutional Neural Network for MNIST

数据增强

- 特征标准化（Feature Standardization）
  - 将每个像素值减去图像的均值，然后除以图像的标准差，以使图像的像素分布接近零均值和单位方差。
- ZCA白化（ZCA Whitening）
- 随机旋转、平移、翻转
- Keras图像增强API
  - ImageDataGenerator

```
datagen = ImageDataGenerator()
```



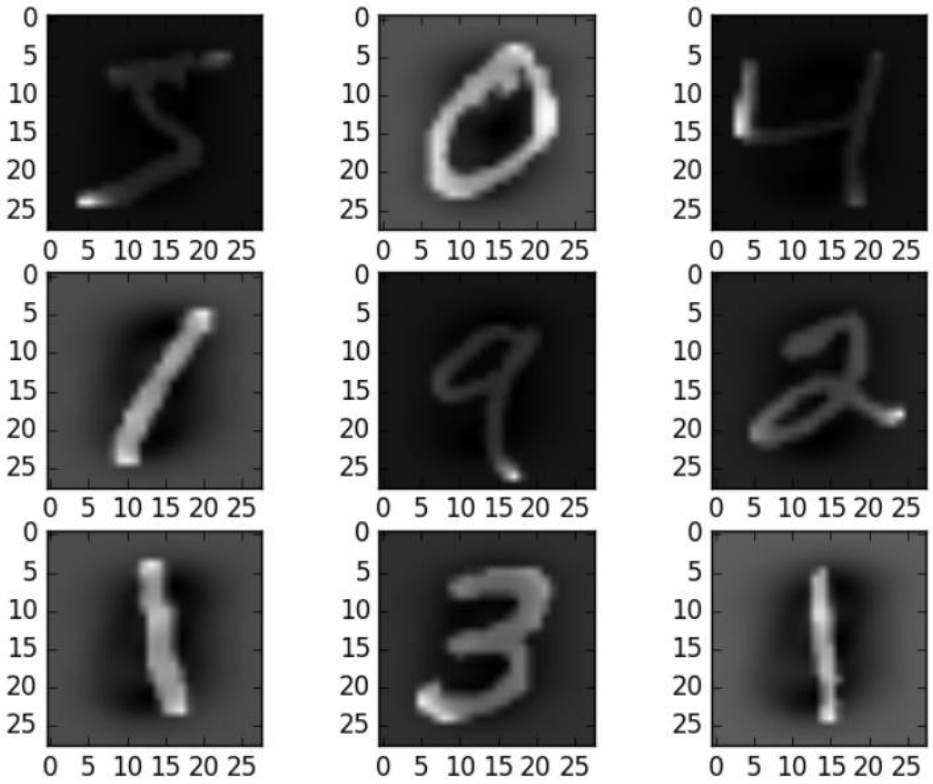
←

记住数据集原本的样子！
接下来就要发生一些变化啦！

特征标准化

- 特征标准化
  - 在 ImageDataGenerator 类中设置 featurewise_center 和 featurewise_std_normalization

```
datagen = ImageDataGenerator(featurewise_center=True, featurewise_std_normalization=True)
```

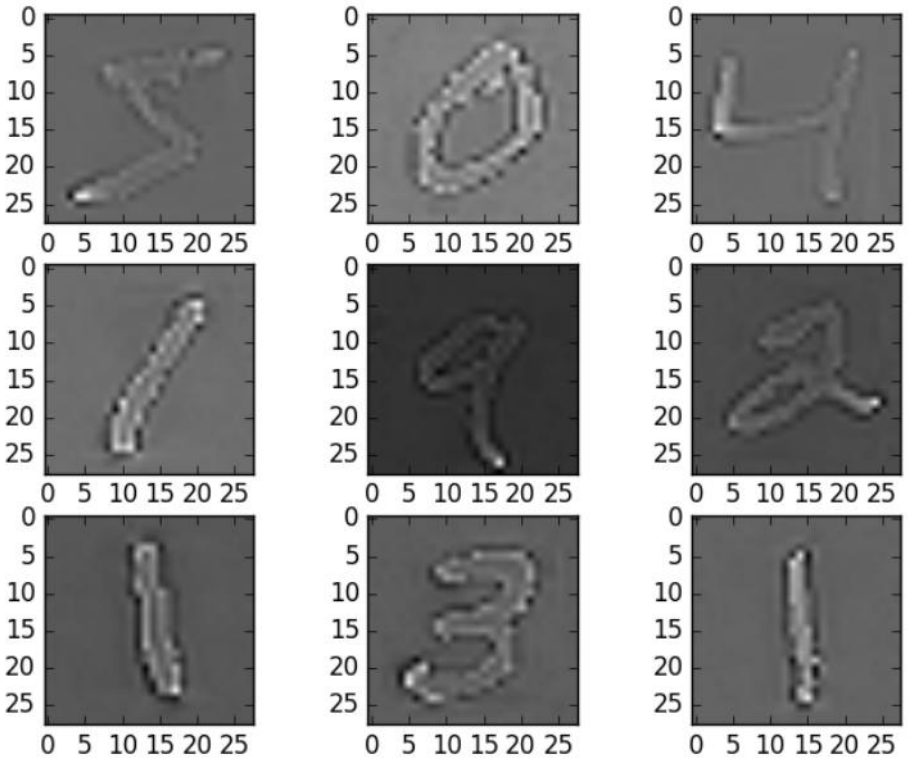


ZCA白化

ZCA白化

- 图像的白化变换是一种线性代数运算，它可以减少像素图像矩阵中的冗余度。
- 在 ImageDataGenerator 类中设置 zca_whitening

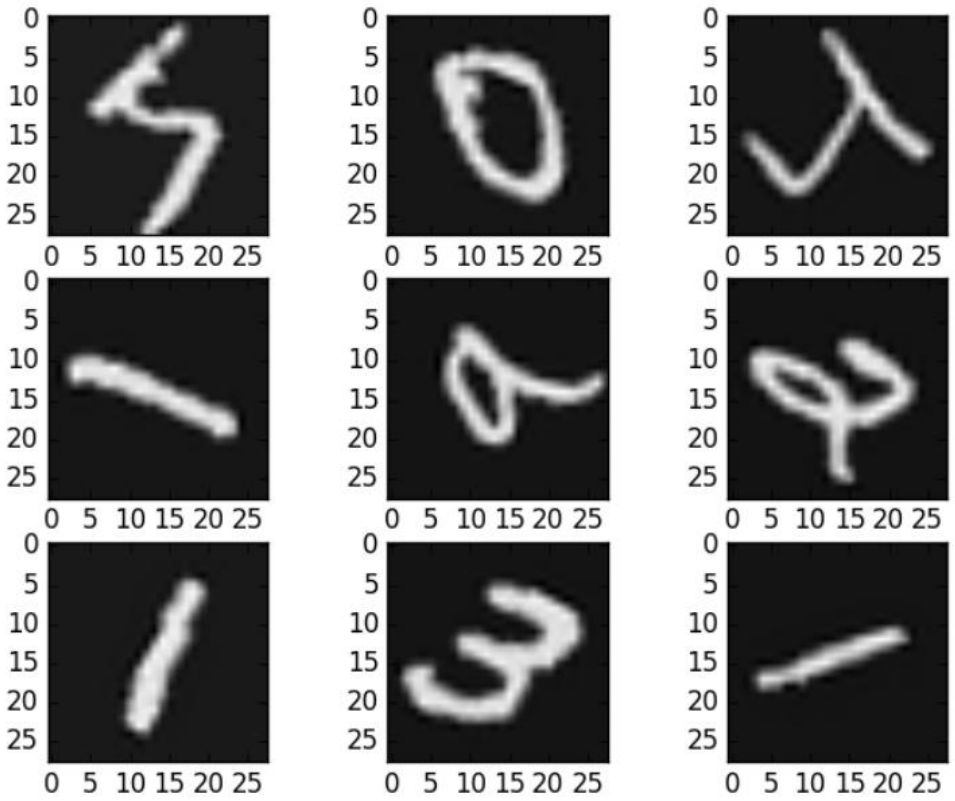
```
datagen = ImageDataGenerator(zca_whitening=True)
```



随机旋转

- 随机旋转
  - 在 ImageDataGenerator 类中设置 rotation_range

```
datagen = ImageDataGenerator(rotation_range=90)
```

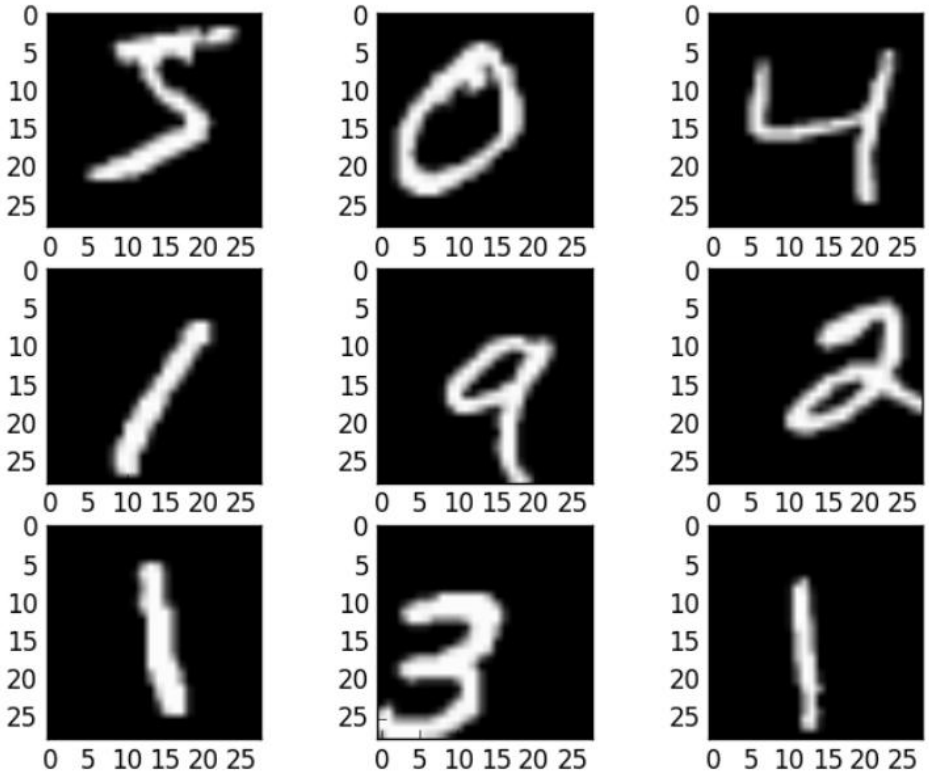


随机平移

随机平移

- 在 ImageDataGenerator 类中设置 width_shift_range 和 height_shift_range

```
datagen = ImageDataGenerator(width_shift_range=shift, height_shift_range=shift)
```



随机翻转

随机翻转

- 在 ImageDataGenerator 类中设置 vertical_flip 和 horizontal_flip

```
datagen = ImageDataGenerator(horizontal_flip=True, vertical_flip=True)
```



5 循环神经网络(RNN)

RNN

卷积神经网络
(CNN)

循环神经网络（RNN)

具有记忆的神经网络

RNN

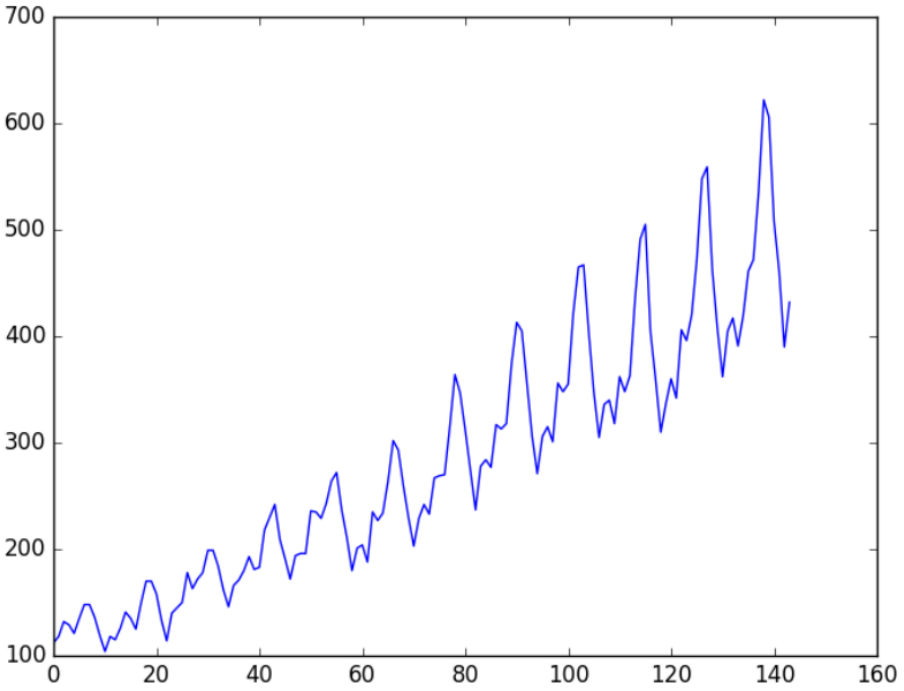
- 概述
  - 循环神经网络（RNN）是一种非常灵活的神经网络结构，适用于处理各种类型的序列数据。
- 一对多（One-to-Many）
  - 对于图像字幕生成（image captioning）任务，RNN可以接收一张图像作为输入，并逐步生成描述图像内容的文字序列。在这种情况下，RNN的输出序列的长度可变，因为它需要根据图像内容逐步生成不同长度的描述。
- 多对一（Many-to-One）
  - 对于情感分类（sentiment classification）任务，RNN可以接收一个由单词组成的序列作为输入，并输出该序列所表达的情感类别。在这种情况下，RNN将输入的单词序列转换为一个固定长度的向量表示，然后对其进行分类。
- 多对多（Many-to-Many）
  - 对于机器翻译（machine translation）任务，RNN可以接收一个语言的序列作为输入，并输出另一个语言的序列。在这种情况下，RNN需要同时处理输入序列和输出序列，以便将源语言的内容正确翻译成目标语言。
- 同步多对多（Synchronized Many-to-Many）
  - 对于视频分类（video classification）任务，RNN可以接收一个视频序列作为输入，并输出每一帧视频所对应的类别。在这种情况下，RNN需要同时处理输入的视频序列和输出的类别序列，并确保它们之间的时间同步性。

时间序列预测

- 任务描述
  - 国际航空公司的乘客预测问题。给定年份和月份，任务是预测国际航空乘客数量，单位为千。

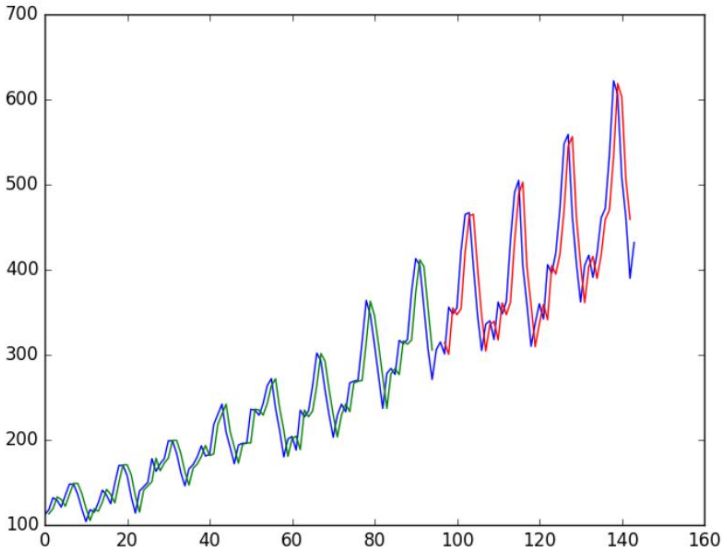
```
"Month","Passengers"
"1949-01",112
"1949-02",118
"1949-03",132
"1949-04",129
"1949-05",121
```

数据集可视化



基于MLP的时间序列预测

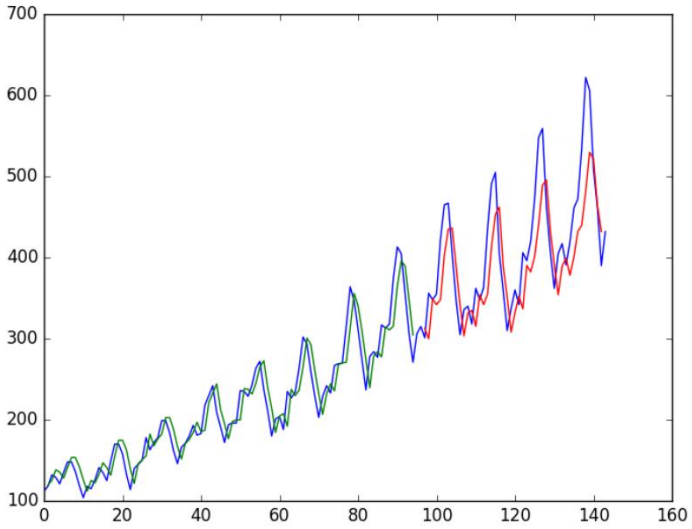
```
30  # create and fit Multilayer Perceptron model
31  model = Sequential()
32  model.add(Dense(8, input_dim=look_back, activation='relu'))
33  model.add(Dense(1))
34  model.compile(loss='mean_squared_error', optimizer='adam')
35  model.fit(trainX, trainY, epochs=200, batch_size=2, verbose=2)
36  # Estimate model performance
37  trainScore = model.evaluate(trainX, trainY, verbose=0)
38  print('Train Score: %.2f MSE (%.2f RMSE)' % (trainScore, math.sqrt(trainScore)))
39  testScore = model.evaluate(testX, testY, verbose=0)
40  print('Test Score: %.2f MSE (%.2f RMSE)' % (testScore, math.sqrt(testScore)))
41  # generate predictions for training
42  trainPredict = model.predict(trainX)
43  testPredict = model.predict(testX)
```



蓝色＝全数据集
绿色＝训练
红色＝预测

基于LSTM循环神经网络的时间序列预测

```
# create and fit the LSTM network
model = Sequential()
model.add(LSTM(4, input_shape=(1, look_back)))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(trainX, trainY, epochs=100, batch_size=1, verbose=2)
# make predictions
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# invert predictions
trainPredict = scaler.inverse_transform(trainPredict)
trainY = scaler.inverse_transform([trainY])
testPredict = scaler.inverse_transform(testPredict)
testY = scaler.inverse_transform([testY])
# calculate root mean squared error
trainScore = math.sqrt(mean_squared_error(trainY[0], trainPredict[:,0]))
print('Train Score: %.2f RMSE' % (trainScore))
testScore = math.sqrt(mean_squared_error(testY[0], testPredict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
```



蓝色＝全数据集
绿色＝训练
红色＝预测

6 编程实践

厦门大学数字实验平台

- 网址：http://10.26.57.15
- 课程邀请码：WSYLC

End

